2 Open problems in mechanistic interpretability methods and foundations

One way of thinking about what neural networks do internally is that they learn parameters that implement neural algorithms. These neural algorithms take data as input and, through a series of steps, transform their internal activations to produce an output. Different parts of the network learn different steps in the algorithm; mechanistic interpretability aims to describe the function of different network components.

Broadly speaking, there are two methodological approaches to achieving this. The first approach, often called 'reverse engineering', is to decompose the network into components and then attempt to identify the function of those components (Section 2.1). This approach "identifies the roles of network components". Conversely, the second approach, sometimes referred to as 'concept-based interpretability', proposes a set of concepts that might be used by the network and then looks for components that appear to correspond to those concepts (Section 2.2). This approach thus "identifies network components for given roles".

In this section, we will examine both approaches ('Reverse engineering' and 'Concept-based interpretability') (Figure **1**), discussing their methods and open problems. We will also touch on open problems that cut across either approach, including proceduralizing the mechanistic interpretability pipeline (Section 2.3) and its uses in automating interpretability research (Section 2.4).

2.1 Reverse engineering: Identifying the roles of network components

2.1.1 Reverse engineering is necessary because AI and humans use different representations and perform different tasks

Large language models produce text that closely resembles human writing, and it is tempting to assume that they generate it through cognitive processes similar to those of human writers[1]. However, humans and AI models often solve problems in different ways. For example, a model that is only 1% the size of GPT-3 outperforms humans on next-token prediction tasks (Shlegeris et al., 2024). Inversely, even state-of-the-art multimodal LLMs struggle with tasks that a four-year-old could easily master, such as learning causal properties of new objects involving simple lights and shapes (Kosoy et al., 2023). An even clearer sign that humans and AI are using different representational processes are cases where humans cannot solve a problem at all, as in the case of predicting a protein structure from sequence (Jumper et al., 2021; Lin et al., 2023).

Even when both humans and AI exhibit comparable levels of competence on a given task, they may use different heuristics. For example, research shows that image models tend to rely more heavily on textural features (such as recognizing elephants by their hide rather than their shape (Geirhos et al., 2019)) or rely on dataset correlations (as when identifying fish by the fingers that proud fisherman use to hold them (Brendel & Bethge, 2019)) to a greater degree than people do. Even simple algorithmic tasks, like modular addition, which humans might solve with simple carries, were solved by a small transformer model by learning a Fourier transform strategy that researchers only understood in retrospect (Nanda et al., 2023a).

To grasp the potentially alien cognition of these models, we must develop methods to uncover and understand the previously unknown concepts and mechanisms implemented within them. In other words, we must be able to reverse engineer these models (Olah, 2024).

Reverse engineering generally involves three steps, whether it is an engine, a piece of software, or a neural network (Figure **2**):

1. **Decomposition**: Breaking down the object of study into simpler parts (Section 2.1.2);
2. **Description of components**: Formulating hypotheses about the functional role of component parts and how they interact (a process that can be called 'interpretation') (Section 2.1.3);
3. **Validation of descriptions**: Testing if our hypotheses are correct (Section 2.1.4).

[1] However, even if this assumption were true, understanding LLMs would remain challenging, as we also lack a mechanistic understanding of the cognitive processes involved in human text creation!

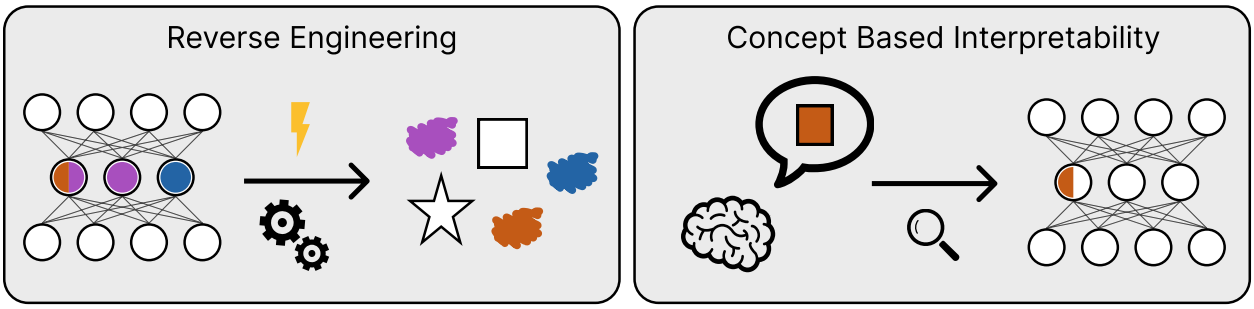

Figure 1: Two approaches to neural network interpretability. (Left) Reverse Engineering is characterized by decomposing networks into functional components and describing how those components interact to produce the network's behavior. It thus aims to 'identify the roles of network components' (Section 2.1). (Right) Concept-based interpretability on the other hand attempts to discover human concepts within neural network internals. It thus aims to 'identify the network components for given roles' (Section 2.2).

If our hypotheses are invalidated, we must either improve our decomposition or improve our hypotheses regarding the functional roles of its components. We will systematically examine each step, analyzing current methods and their respective shortcomings.

2.1.2 Reverse engineering step 1: Neural network decomposition

In mechanistic interpretability, our aim is to decompose a neural network and study its parts in isolation in order to explain how neural networks generalize. This aim raises the question of how best to carve a neural network "at its joints" for the purposes of interpretability.

2.1.2a Networks do not naturally decompose into architectural components.

The naive approach to decompose neural networks involves breaking them down into their architectural components, such as individual neurons, attention heads, or layers.

Some early attempts to interpret deep artificial neural networks studied the responses or weight structure of individual neurons or single convolutional filters (Erhan et al., 2009; Le et al., 2012; Krizhevsky et al., 2012; Szegedy et al., 2014; Simonyan et al., 2014b; Zhou et al., 2015; Srivastava et al., 2014; Yosinski et al., 2015; Mordvintsev, 2015; Olah et al., 2017a; Bau et al., 2020; Dalvi et al., 2019; Olah et al., 2020a; Cammarata et al., 2020). These efforts paid homage to the 'Neuron Doctrine' in neuroscience, which posits that individual neurons are the structural and functional unit of the nervous system (Cajal, 1924; Sherrington, 1906). However, researchers discovered that single neurons are 'polysemantic' – they seem to respond to multiple kinds of features in both artificial (Wei et al., 2015; Nguyen et al., 2016c; Olah et al., 2017a) and biological networks (Churchland & Shenoy, 2007; Rigotti et al., 2013; Mante et al., 2013; Raposo et al., 2014). These observations support earlier theoretical work that suggested representations used by neural networks do not necessarily align with the activation of individual neurons (Hinton, 1981).

Interpreting individual attention heads does not fare better than interpreting individual neurons, as attention heads also exhibit polysemanticity (Janiak et al., 2023). More broadly, research suggests that studying the attention patterns of models can often be misleading (Jain & Wallace, 2019; Pruthi et al., 2020).

Some work even suggests that representations in language models might span multiple layers (Yun et al., 2021; Lindsey et al., 2024). This chimes with work that edits or intervenes on individual layers, which indicates that this level is too coarse-grained to robustly carve the network at its joints (Meng et al., 2022b; Wang et al., 2023).

If natural architectural components, such as individual neurons, attention heads, or layers, do not provide a natural way to decompose neural network representations, then what does?

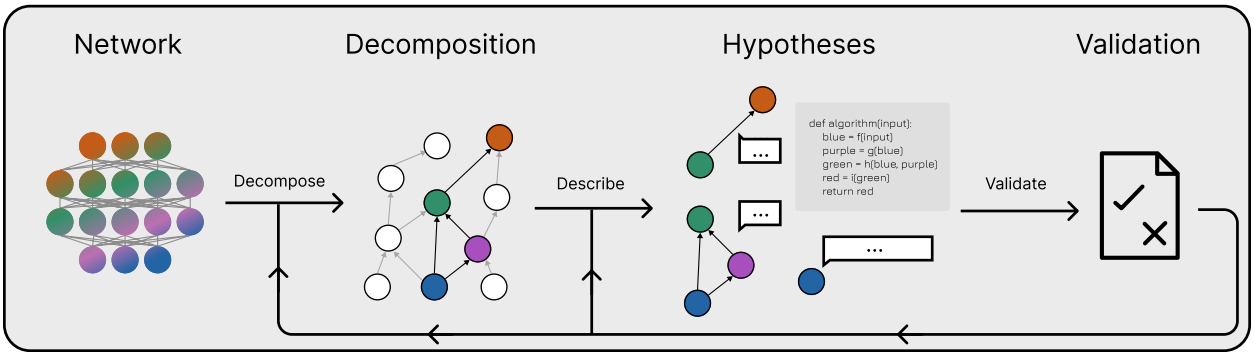

Figure 2: The steps of reverse engineering neural networks. (1) Decomposing a network into simpler components. This decomposition might not necessarily use architecturally-defined bases, such as individual neurons or layers (Section 2.1.2). (2) Hypothesizing about the functional roles of some or all components (Section 2.1.3). (3) Validating whether our hypotheses are correct, creating a cycle in which we iteratively refine our decompositions and hypotheses to improve our understanding of the network (Section 2.1.4).

2.1.2b Decomposition by dimensionality reduction methods.

If individual neurons are not the right decomposition, perhaps groups or patterns of neurons are. Many decomposition methods attempt to identify activation vectors that correspond to the basic unit of neural network computation. One common approach is to provide models with a range of unlabeled inputs, collect the resulting hidden activations, and then apply unsupervised dimensionality reduction techniques to these hidden activations. The hope is that structure in the hidden activations corresponds to the structure of neural computation. Commonly used dimensionality reduction methods include Principal Component Analysis or Singular Value Decomposition (Hollinsworth et al., 2024; Marks & Tegmark, 2024; Huang et al., 2024a; Bushnaq et al., 2024) and non-negative matrix factorization (Olah et al., 2018; Voss et al., 2021; Cammarata et al., 2020), though these techniques are no longer predominant methods used for mechanistically decomposing language models (Friedman et al., 2024).

2.1.2c Decomposition by sparse dictionary learning (SDL).

According to the 'superposition hypothesis', neural networks are capable of representing more features than they have dimensions, as long as each feature activates sparsely (Elhage et al., 2021) (Figure **3**). This is a key reason that dimensionality reduction methods are not considered state-of-the-art, because they cannot identify more directions than there are activation dimensions. The superposition hypothesis, coupled with the failure of dimensionality reduction methods to overcome it, motivated the search for methods that can identify more directions than dimensions. Recent work has explored the use of sparse dictionary learning (SDL) to this end (Elhage et al., 2021; Sharkey et al., 2022b; Huben et al., 2024; Bricken, 2023).

Currently, SDL is the most popular set of unsupervised decomposition methods in mechanistic interpretability. SDL encapsulates a family of methods, including Sparse Autoencoders (SAEs) (Gao et al., 2024; Templeton et al., 2024; Rajamanoharan et al., 2024; Makelov et al., 2024; Kissane et al., 2024b; Braun et al., 2024), Transcoders (Dunefsky et al., 2024), and Crosscoders (Lindsey et al., 2024).

In SDL, hidden activations are typically passed to a small neural network consisting of only two layers, which correspond to an encoder and decoder respectively, with a wide hidden space. The encoder activations represent how active each 'latent' [2] is, and the decoder matrix corresponds to a dictionary of latent directions. We want to train the dictionary elements to align with 'feature directions' in the network's hidden activations. Since we assume that individual features are sparsely present in the activations, the encoder activations are trained to be sparsely activating. In the case of SAEs, the output is trained to either reconstruct the

[2]The term latent is often used instead of the word feature to refer to SDL dictionary elements, since the term 'feature' is often used to refer to multiple different ideas (Smith, 2024b)